16/9/20 Gate Coby-2
C Programming Cont.

Date
Page 1

→ As scanf & gets functions are risky to use. Hence, it is advisable to design our own I/P function.

We want our I/P function to have full functionality
1) It must continue to read the string even after seeing white space characters
2) It must stop reading the string after seeing the newline character.
3) It must discard extra characters.
4) And, it must return the number of characters it stores in the character array.

getchar() function is used to read one character at a time from the user I/P. It returns an integer equivalent to the ASCII code of the character
↳ That's why we have written int ch not char ch.

```
#include <stdio.h>
int input(char str[], int n){
    int ch, i=0;
    while((ch=getchar()) != null){
        if(i<n)
            str[i++]=ch;
    } str[i]='\0';
    return i;
}
int main(){
    char str[100];
    int n=input(str,5);
    printf("%d %s", n, str);
}
```

I/P: Hello, How are you?
O/P: 5 Hello

Scanned with CamScanner

→ putchar() in C

Prototype: int putchar(int ch)

putchar accepts an integer argument (which represents a character, it writes to stdout and returns an integer representing the character written on the screen)

Why int ch not char ch?
↳ Always remember that character is internally represented in binary form only. It doesn't make any difference if you write int ch instead of char ch.

eg #include <stdio.h>
int main() {
Int ch;
for (ch='A'; ch<='Z'; ch++)
putchar(ch);
return 0;
}

O/P: ABCDEFGHIJKLMNOPQRSTUVWXYZ

O→ which didn't work properly)
a) printf("%c", '\n'); //✓ Expected 'give char
b) putchar('\n'); //✓ Expected char provided

c) putchar('\n'); // ✓ Given provided char
d) putchar("\n"); // ✗ Extended character give string
e) puts('\n'); // ✗ expected string give char
f) puts("\n"); // ✓
g) printf("%c", '\n'); ✗
h) " " " , "\n"); ✓

Introduction to C string library & strcpy (string copy) function

→ There are some operations which can be performed on strings. For eg: copy strings, concatenate strings, select substrings and so on.

<string.h> library contains all the required functions for performing string operations. So we just have to include the header file: #include<string.h> in our program and we are good to go.

Strcpy (String copy) function

Prototype: char* strcpy(char* destination, const char* source)

strcpy is used to copy a string pointed by source (including NULL character) to the destination (character array).

strcpy → returns the pointer to first character of the string which is copied in destination.

Hence if we use %s then whole string will be printed on the screen.

Scanned with CamScanner

eg
```
#include <stdio.h>
#include <string.h>
int main(){
    char str1[10] = "Hello";
    char str2[10];
printf("%s\n", strcpy(str2, str1));
printf("%s", str2);
return 0;
}
```
o/p: Hello
Hello

We can also chain together a series of strcpy calls

```
#include <stdio.h>
#include <string.h>
int main(){
    char str1[10] = "Hello";
    char str2[10];
    char str3[10];

strcpy(str3, strcpy(str2, str1));
printf("%s %s", str2, str3);
return 0;
}
```

o/p Hello Hello

Scanned with CamScanner

In the call to strcpy(str1, str2) there is no way the strcpy will check whether the string pointed by str2 will fit in str1.

If the length of the string pointed by str2 is greater than the length of the character array str1 then it will be an undefined behaviour

To avoid this, we can use strncpy function.

strncpy(destination, source, sizeof(destination));

Eg
```
#include<stdio.h>
#include<string.h>
int main(){
  char str1[6]="Hello";
  char str2[4];
  strncpy(str2, str1, sizeof(str2));
  printf("%s", str2); // Hell
  return 0;
}
```

strncpy will copy the string in str2 (destination) without a terminating NULL character, if the size of str1(source) is equal to or greater than the size of str2(destination). To do that eg →

```
#include<stdio.h>
#  ——<string.h>
int main(){
  char str1[6]="Hello";
  char str2[6]; strncpy(str2, str1, sizeof(str2));
  str2[sizeof(str2)-1]='\0';
  printf("%s", str2);
```

Scanned with CamScanner

strlen (String length) function

Prototype size_t strlen(const char *str);

size_t is an unsigned integer type of at least 16 bits.

strlen() is used to determine the length of the given string

To the strlen function, we should pass the pointer to the first character of the string whose length we want to determine

And as a result, it returns the length of the string.

Note: It doesn't count NULL character.

```
#include<stdio.h>
#include<string.h>
int main(){
    char *str = "Hello world";
    printf("%d", strlen(str));
    return 0;
}
```

// //

Also in ① if we write

char str[] = "Hello world";

O/P //

if char str[100] ⇒ O/P //

Scanned with CamScanner

strcat (String concatenation)

Prototype: char* strcat(char* str1, const char* str2);

Strcat function appends the content of string str2 at the end of string str1.

It returns the pointer to resulting string str1

Eg 1
```
#include <stdio.h>
#include <string.h>
int main(){
    char str1[100], str2[100];
    strcpy(str1, "Welcome to ");
    strcpy(str2, "our Academy");
    strcat(str1, str2);
    printf("%s", str1);
}
```

O/P: Welcome to our Academy

Caution-

An undefined behaviour can be observed if size of str1 isn't long enough to accomodate the additional character of str2. For eg

```
#include <stdio.h>
#include <string.h>
int main(){
    char str1[15], str2[100];
    strcpy(str1, "Welcome to ");
    strcpy(str2, "our Academy"); strcat(str1, str2);
    printf("%s", str1);
}
```

O/P
Undefined

→ Strncat is safer version of strcat
It appends the limited number of
characters specified by the third
argument passed to it.

Note: strncat automatically adds NULL
character at the end of the resultant
string.

```
int main() {
Char str1[5], str2[100];
strcpy (str1, "He");
strcpy (str2, "llo!");
strncat (str1, str2, sizeof (str1) - strlen(str1) - 1);
printf ("%s", str1);
return 0;
}
```

5 − 2 − 1 = 2 (for NULL)

O/P: Hell

strcmp(String comparison) function

Prototype: int strcmp(const char* s1, const char*

→ Compares 2 strings s1 & s2
→ Returns value
less than 0, if s1 < s2
Greater than 0, if s1 > s2
equal to 0, if s1 == s2

Scanned with CamScanner

Attention

In ASCII character set →

a) All upper case letters are less than all the lower case letters (upper case letters have ASCII codes b/w 65 & 90 & lower case letters have ASCII codes b/w 97 & 122)

b) Digits are less than letters (0-9 '22) digits have ASCII codes 48 to 57)

c) Spaces are less than all printing characters (space character has the value 32 in ASCII set).

strcmp considers s1 < s2 if

a) when first i characters in s1 & s2 are same & (i+1)st character of s1 is less than s2.

Eg:
```
#include<stdio.h>
#include<string.h>
int main(){
    char *s1="abcd";
    char *s2="abce";
    if(strcmp(s1,s2)<0)
    printf("s1 is less than s2");
    else
    printf("s1 is greater than or equal to s2");
    return 0;
}
```

o/p: s1 is less than s2.

Scanned with CamScanner

i) can be 0 also in case a)

b) All characters of s1 match s2, but s1 is shorter than s2.

```
#include<stdio.h>
#include<string.h>
int main(){
  char *s1 = "abc";
  char *s2 = "abcd";
  if (strcmp(s1,s2) < 0)
  printf("s1 is less than s2");
  else
  printf("s1 is greater than or equal to s2");
  return 0;
}
```

O/P: s1 is less than s2

Array of strings.

```
char[][12] = {"2 Oranges", "2 Apples", "3 Bananas", "1 Pineapple"};
```

2		O	r	a	n	g	e	s	10	10	10
2		A	p	p	l	e	s	10	10	10	10
3		B	a	n	a	n	a	s	10	10	10
1		P	i	n	e	a	p	p	l	e	10

Draw back → A lot of memory is wasted

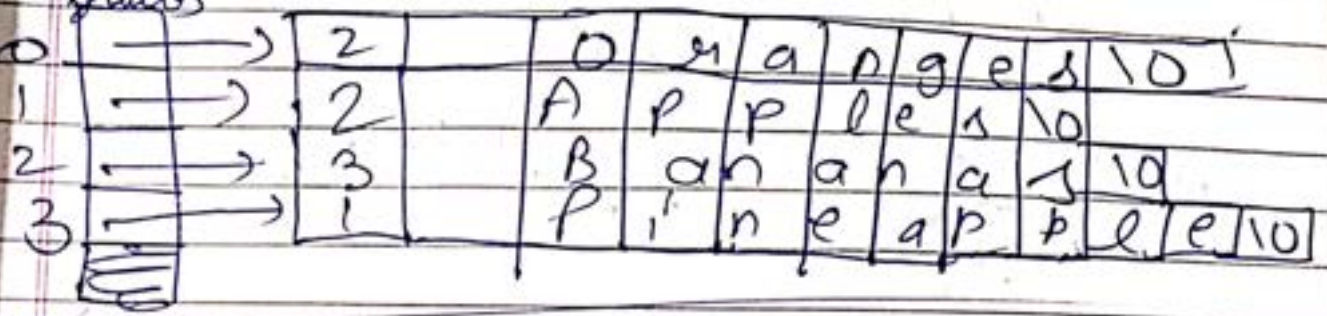
Don't worry!

We have an alternative

Scanned with CamScanner

Instead of making 2-D array, we make array of pointers

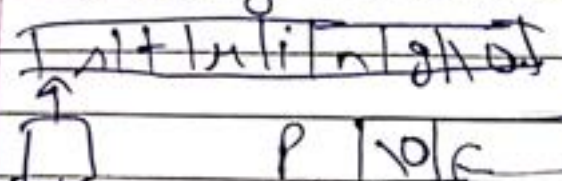
char *fruits[] = {"2 Oranges", "2 Apples", "3 Bananas", "1 Pineapple"};



Gate 2004

Q) 1) char p[20];
char *s = "string";
int length = strlen(s);
int i;
for (i=0; i<length; ++i)
p[i] = s[length-i];
printf("%s", p);

O/P a) gnirts c) gnirt
b) string d) None.



∴ length = 6 s[length-i] = \0

So NO O/P

Scanned with CamScanner

Gate 2011

Q. char c[] = "GATE2011";
char *p = c;
printf("%s", p + p[3] - p[1]);
a) GATE 2011
b) E2011
c) 2011 d) 011

Ans c

	G	A	T	E	2	0	1	1	\0
1000	1	2	3	4	5	6	7	8	

p

p = 1000, p[3] = E
p[1] = A

So printf("%s", 1000 + E - A);

Always remember that character is internally represented as integer only.

1000 + 69 - 65, 1000 + 4

So 10004 → 2

So 2011

Scanned with CamScanner

Qn 2015

Q - Consider the foll^n function written in C. Th
O/P of function on input "ABCD EFGH" is

```
void foo(char *a){
if(*a != ' '){
    foo(a+1);
    putchar(*a);
}}
```

a) ABCD EFGH
b) ABCD
c) HGFE DCBA
d) DCBA

Hint: Remember recursion

Q -

```
void fun1(char *s1, char *s2){
char *tmp;
tmp = s1;
s1 = s2;
s2 = tmp;}
void fun2(char **s1, char **s2){
char *tmp;
tmp = *s1;
*s1 = *s2;
*s2 = tmp;
}
int main(){
char *str1 = "Hi", *str2 = "Bye";
fun1(str1, str2); printf("%s %s", str1, str2);
fun2(&str1, &str2); printf("%s %s", str1, str2);
}
```

a) Hi Bye Bye Hi
b) Hi Bye Hi Bye
c) Bye Hi Hi Bye
d) Bye Hi Bye Hi

Scanned with CamScanner

Gate 2017

Q. O/P of

```
#include<stdio.h>
#include<string.h>
int main(){
  char *c = "GATECSIT2017";
  char *p = c;
  printf("%d", (int)strlen(c+2[p]-6[p]-1));
}
```

Suppose &c = 1000

Ans 2[p] = *(2+p) = *(1002) = T

6[p] = *(6+p) = I

T - I - 1 = 19 - 1 - 1 = 10

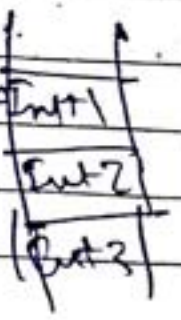
(int)sizeof(1000 + 10) = 2 (∵ 1010 is 10, 1011 is 7 & 1012 is 10)

Type casting size_t → int

Function pointers

Def^{n} → These are like normal pointers but they have the capability to point to a function.

```
int fun(int a, int b)
{
return a+b;
}
```



Inst - Instruction

Scanned with CamScanner

Eg How to declare a pointer to array

```
int main()
{
int *ptr[10]; // wrong because [] precedence > *
return 0;
}
```

So ptr is an array of 10 pointer pointing to integers.

So, `int (*ptr)[10]; //ptr is a pointer pointing to 10 integers`
or ptr pointing to array of 10 int ger

How to declare a pointer to function?

Eg I want a pointer to

```
int add(int a, int b)
{return a+b;}
```

```
int main(){
  int (*ptr)(int, int) = &add;
}
```

or `int (*ptr)(int, int);`
`ptr = &add;`

```
int main(){
int result;
int (*ptr)(int, int) = &add; OR *ptr = add
result = (*ptr)(10,20);      = ptr(10,20);
printf("%d", result);
}
```

automatically address of fun

Scanned with CamScanner

Application-function Pointers

Suppose we want to call a function named fun() at a certain point of time in our code.

```
int func(int a, int b)
{return a+b;}
int main(){
printf("%d", fun(2,5));
}
```

// There is nothing has to be decided at runtime.

There ~~has~~ are some situations in which user has to decide which function has to be called at a particular point of time.

Let say, we want to design a calculator which has the capability to perform addition, subtraction, multiplication & division.

Here, the user will decide which operation he/she wants to perform.

Suppose, we have decided to create separate functions for these operations.

Now, we want user to decide which function has to be called at runtime.

Scanned with CamScanner

→ one way is to use if/switch expression

eg
```
#include <stdio.h>
float sum(float a, float b){return(a+b);}
float sub(float a, float b){return (a-b);}
float mult(float a, float b){return (a*b);}
float divi(float a, float b){return (a/b);}

int main(){
  int choice;
  float a, b, result;
  printf("Enter your choice: 0 for sum, 1 for sub, 2
  for mult, 3 for div: \n");
  scanf("%d", &choice);
  printf("Enter the two numbers: \n");
  scanf("%f %f", &a, &b);

switch(choice){
  case 0: result = sum(a,b); break;
  case 1: result = sub(a,b); break;
  case 2: result = mult(a,b); break;
  case 3: result = divi(a,b); break;
}
printf("%f", result);
return 0;
}
```

Scanned with CamScanner

Another way is to use function pointer

```
#include<stdio.h>
#define ops 4

float sum(float a, float b) {return (a+b);}
float sub(float a, float b) {return (a-b);}
float mult(float a, float b) {return (a*b);}
float div(float a, float b) {return (a/b);}

int main()
{
float (*ptr2func[ops])(float, float) = {sum, sub,
                                        mult, div};
int choice;
float a, b;
printf("Enter your choice: 0 for sum, 1 for
sub, 2 for mult, 3 for div: \n");
scanf("%d", &choice);
printf("Enta the two numbers: \n");
scanf("%f %f", &a, &b);
printf("%f", ptr2func[choice](a, b));
}
```

Introduction to Structure

- I have a garage
- I want to store all the info about cars which are available in my Garage.
  - Engine
  - Fuel Type
  - Fule Tank Capacity
  - Seating Capacity
  - City Mileage

Scanned with CamScanner

1) 1 way to create variables, Ordinary
   1 car - 5 variables
   2 car - 10 variables.

   So time & memory consuming

2) Array is also not a good solution
   Array has the capability to store more than one elements but they all must be of same type.

3) Structure is one stop solution
   Structure - A structure is a user defined data type that can be used to group elements of different types into a single type.

Declaring structure →

Defn of structure → A structure is a user defined data type that can be used to group elements of different types into a single type.

Syntax

```
struct {
    char *engine;
    char *fuel_type;
    int fuel_tank_cap;
    int seating_cap;
    float city_mileage;
} car1, car2;
```

variables

eg-

```
struct {
  char *engine;
} car1, car2;
int main() {
car1.engine = "DDis 190 Engine";
car2.engine = "1.2 L Kappa Dual VTVT";
printf("%s\n", car1.engine);
printf("%s", car2.engine);
}
```

O/P DDis 190 Engine
1.2 L Kappa Dual VTVT

Structure Tyber - Using Structure Tag

```
struct {          // This structure is in global
  char *engine;   // scope. Hence, visible to all
};                //          functions.
int main() {
  car1.
}
```

what if we want structure in local scope, let's see through eg →

```
#include <stdio.h>
struct {
    char *name;
    int age;
    int salary;
} emp1, emp2;
```

Scanned with CamScanner

```
int manager()
struct {
    char name;
    int age;
    int salary;
} manager;
manager.age = 27;
if (manager.age > 30)
    manager.salary = 65000;
else
    manager.salary = 55000;
return manager.salary
}
int main(){
    printf("Enter the salary of employee1: ");
    scanf("%d", &emp1.salary);
    printf("Enter the salary of employee2: ");
    scanf("%d", &emp2.salary);
printf("Employee 1 salary is: %d\n", emp1.salary);
printf("Employee 2 salary is: %d\n", emp2.salary);
printf("Manager's salary is %d", manager());
return 0;
}
```

- Record scope (inside a function)

- In this example, we have seen that we have to write same piece of code again

Alter → P.T.O.

Scanned with CamScanner

```
struct employee {
    char *name;
    int age;
    int salary;
};
int manager() {
struct employee manager; //
}
int main() {
    struct employee emp1;
    struct employee emp2;
}
```

eg:-

```
#include<stdio.h>
struct employee {
    char name;
    int age;
    int salary;
};
int manager() {
struct employee manager;
manager.age=27;
```



```
return 0;
}
```

Scanned with CamScanner

Structure types - using typedef 23

Syntax: typedef existing_data_type new_data_type

typedef gives freedom to the user by allowing them to create their own types.

```
#include <stdio.h>
typedef int INTEGER;
int main(){
    INTEGER var = 100;
    printf("%d", var);
    return 0;
}
```

Use of typedef in structures.

Structure Declaration

```
struct car{
char engine[50];
Char fuel_type[10];
 int fuel_tank_cap;
int seating_cap;
float city_mileage;
}c1;
```

Separate declaration

```
struct car{
char engine[50];
Char fuel_type[10];
int fuel_tank_cap;
int seating_cap;
float city_mileage;
};
int main(){
  struct car c1;
}
```

Scanned with CamScanner

Use of typedef → old type

```
typedef struct car{
    char engine[50];
    char fuel_type[10];
    int fuel_tank_cap;
    int seating_cap;
    float city_mileage;
} car;                    // New type

int main(){
    struct car C1; //old
}
    Car C1; //new
```

Initializing structure variables & accessing members of structure.

Not allowed
```
struct abc{
    int p=23; } wrong way
    int q=34;
};
```

Allowed
```
struct abc{
    int p;
    int q;
};
int main(){
    struct abc x={23,34};
}
```

Scanned with CamScanner

Accessing members of structure:-
→ We can access members of the structure using dot(.) operator.

eg struct car {
int fuel_tank_cap;
} c1, c2;
int main() {
c1.fuel_tank_cap = 45;
c2.fuel_tank_cap = 30;
printf("%d %d", c1.fuel_tank_cap, c2.fuel_tank_cap);
}

o/p 45 30

Designated Initialization - Allows structure members to be initialized in any order.

struct abc {
int x;
int y;
int z;
};
int main() {
struct abc a = {.y = 20, .x = 10, .z = 30};
printf("%d %d %d", a.x, a.y, a.z);
}

order doesn't matter but don't forget to use dot operator

o/p 10 20 30

Scanned with CamScanner

Declaring an array of structure

Array of structure : Instead of declaring multiple variables, we can also declare an array of structure in which each element of the array will represent a structure variable.

Let's eg →

```
#include<stdio.h>
struct car{
    int fuel_tank_cap;
    int seating_cap;
    float city_mileage;
};
int main(){
    struct car c[2];
    int i;
    for(i=0; i<2; i++){
printf("Enter the car %d fuel tank capacity: ", i+1);
scanf("Enter the car %d seating capacity: ", i+1);
scanf("%d", &c[i].seating_cap);
printf("Enter the car %d city mileage: ", i+1);
scanf("%f", &c[i].city_mileage);
}
printf("\n");
```

Scanned with CamScanner

```
for(i=0; i<2; i++){
    printf("\n Car %d details: \n", i+1);
    printf("Fuel tank capacity: %d\n", c[i].fuel_tank_cap);
    printf("Seating capacity: %d\n", c[i].seating_cap);
    printf("city mileage: %d\n", c[i].city_mileage);
}
}
```

Accessing members of structure using structure pointer

```
struct abc {
    int x;
    int y;
};
int main() {
    struct abc a = {0, 1};
    struct abc *ptr = &a;        — (1)
    printf("%d %d", ptr->x, ptr->y);   (2)
}
```

(1) - ptr is a pointer to some variable of type struct abc.
arrow operator

(2) ptr->x = (*ptr).x // 0

How memory is allocated to structure members?

→ When an object of some structure type is declared then some contiguous block of memory will be allocated to structure members.

Scanned with CamScanner

for eg

```
struct abc, L
    char a;
    char b;
    int c;
} var;
```

What is the size of struct abc?

let int → 4 bytes
Char → 1 byte
So char a = 1, b = 1, c = 4 byte
= 6 byte ✗, Wrong

↳ There is a concept called structure padding

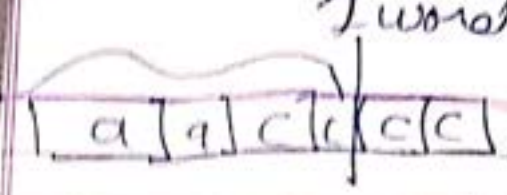
→ Procesor doesn't read 1 byte at a time from memory.
It reads 1 word at a type

↳ If we have 32 bit processor then it means it can access 4 byte at a time which means word size is 4 byte

↳ If we have 64 bit processor then it can access 8 byte at a time which means word size is 8 byte

Problem: In 1 CPU cycle, char a, char b & 2 byte of int c can be accessed
There is no problem with char a & char b
but —

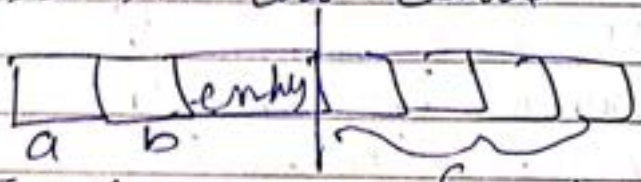
Scanned with CamScanner



Whenever we want the values stored in variable c, 2 cycles are required to access the contents of variable c.
In first cycle, 1st 2 bytes can be accessed & in 2nd cycle, last 2 bytes.

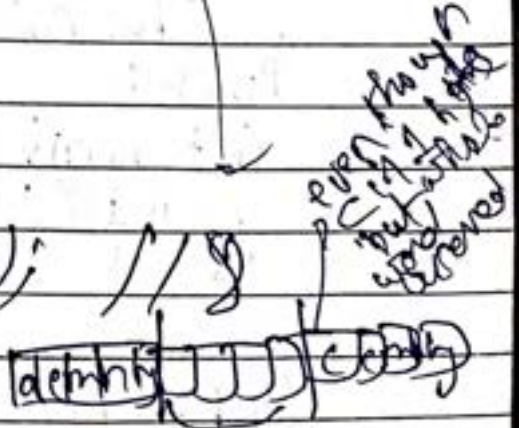
- This an unnecessary wastage of CPU cycles.
- we can save the # of cycles by using the concept called padding.

In 32-bit architecture



Total = 1+1+2+4 bytes = 8 bytes

```
∴ Struct abc{
    char a;
    char b;
    int c;
  };
  int main(){
    struct abc var;
    printf("%d", sizeof(var)); // 8
  }
```



But if char a;
int b;
char c;
o/p = 12

Scanned with CamScanner

17/9/20

Structure Packing in C

→ Because of structure padding, size of structure becomes more than the size of actual structure. Due to this some memory will get wasted.

```
struct abc {
    char a;
    int b;
    char c;
} var;          // 12 size
```

Structure Packing — We can avoid the wastage of memory by simply using #pragma pack(1)

```
#pragma pack(1)
struct abc {
    char a;
    int b;
    char c;
} var;
int main() {
    printf("%d", size of(var));
}
```

O/P = 6

#Pragma is a special purpose directive used to turn on or off certain features

Structure padding

Ans → Either you have to wait for many or more CPU cycles (Structure padding)

Ques 2(1)

```
#include<stdio.h>
struct Q{
    char x, y, z;
};
int main(){
struct Q p={'1', '0', 'a'+2};
struct Q *q = &p;
printf("%c, %c", *((char*)q+1), *((char*)q+2));
}
```

a) 0, c
b) 0, a+2
c) '1', 'a+2'
d) '0', 'c'

Ans → In (2) q is pointing to base address of whole structure & not char. In (3) we are typecasting q to char pointer because we want to print char now and originally q was not of type char (but struct Q)

Ques 2(2)

The following C declarations define s to be:

```
struct node{
int i;
float j;
};
struct node *s[10];
```

a) An array, each element of which is a pointer to a structure of type node
b) A structure of 2 fields, each field being a pointer to an array of 10 elements
c) A structure of 3 fields; an integer, a float and an array of 10 elements
d) An array, each element of which is a structure of type node.

Scanned with CamScanner

Ans-) *s[10]

[ ] has higher precedence, so s is an array not structure.

So b and c is incorrect

Now d says each element of which is a structure of type node but each element is pointer to structure of type node.

So (a) Correct.

Introduction to unions

Unions → Union is a user defined data type but unlike structures, union members share same memory location.

Eg

```
Struct abc {
    int a;
    char b; };
```

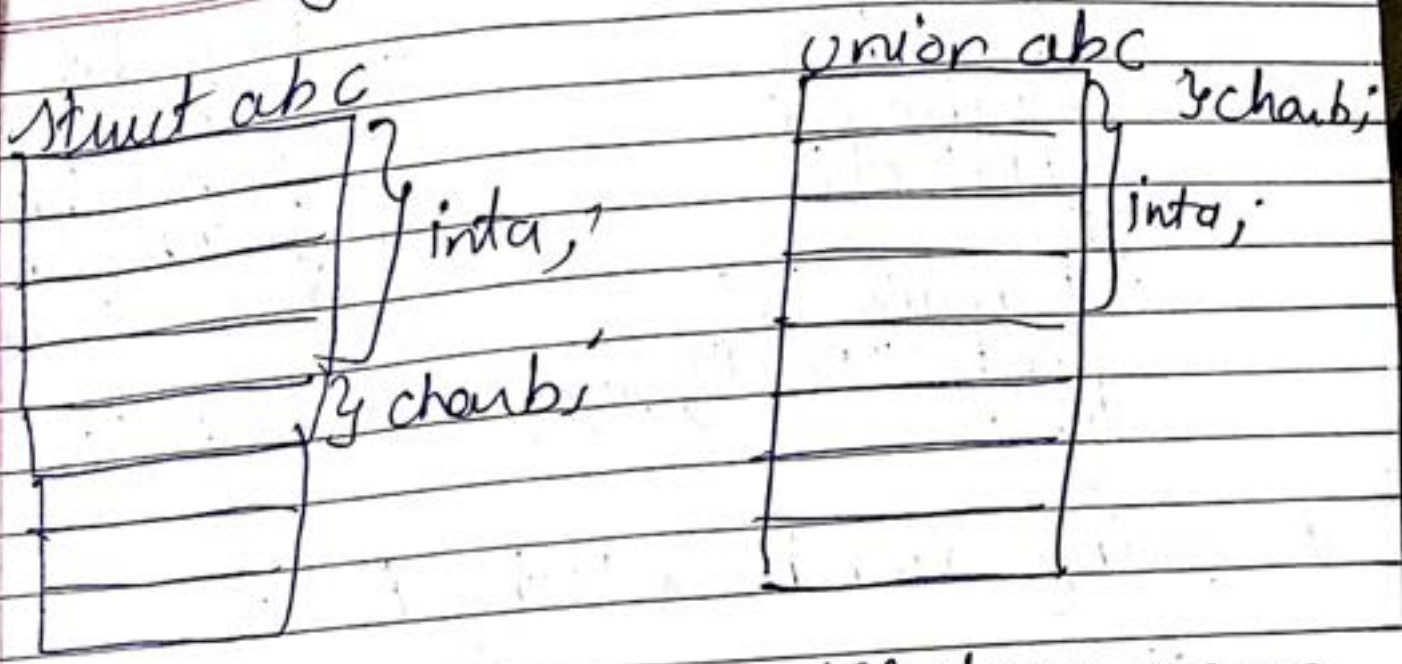
a's address = 6295624
b's " = 6295628

Diff memory locations

```
union abc {
    int a;
    char b;
};
```

a's address = 6295616
b's " = 6295616

Memory allocation



Fact: In union, members will share same memory location. If we make changes in one member then it will be reflected to other members as well.

```
eg union abc {
     int a;
     char b;
   } var;
   int main() {
     var.a = 65;
     printf("a = %d\n", var.a); // 65
     printf("b = %c", var.b); // A
     return 0;
   }
```

Deciding size of union → size of union is taken according to size of larger member of the union.

Scanned with CamScanner

Eg
```
union abc {
    int a;
    char b;
    double c;
    float d;
};
int main() {
    printf("%ld", sizeof(union abc)); // 8
}
```

In my machine
Size of int: 4 bytes
Size of char = 1 byte
Size of double = 8 bytes
Size of float = 4 bytes

Accessing members using Pointers

↳ We can access members of union through pointers by using the arrow (->) operator.

Eg
```
union abc {
    int a;
    char b;
};
int main() {
    union abc var;
    var.a = 90;
    union abc *p = &var;
    printf("%d %c", p->a, p->b);
}
```
// 90 Z

Scanned with CamScanner

Gate 2000

short = 2 bytes
long = 8 bytes
float = 4 bytes

Find memory for variable t
a) 22 bytes b) 14 bytes
c) 18 bytes d) 10 bytes

struct { short s[5]; union { float y; long z; } u; } t;

Applications of union.

→ A store sells 2 kind of items:
1) Books
2) Shirts
Store owner want to keep records of the above mentioned items along with relevant information
Books: Title, Author, number of pages, Price
Shirts: Color, size, design, price

Initially they decided to create a structure like:

```
struct store
{
    double price; //8
    char *title; //8
    char *author; //8
    int num_pages; //4
    int color; //4
    int size; //4
    char *design; //8
};
```

This structure is perfectly usable but only price is common property in both the items & rest are individual.

```
int main()
{
    struct store book;
    book.title = "Jasjot";
    book.author = "Singh";
    book.num_pages = "11";
    book.price = 2000 // dollars
}
```

Size 4 bytes not used by book it is a wastage of memory.

So we can save lot of space if we start using Union.

Scanned with CamScanner

Eg #pragma pack(1)
struct item{
8 bytes → double price;
union {
struct {
20 bytes { char *title;
char *author;
int num_pages;
} book;
16 bytes { struct { int color;
int size;
char *design;
} shirt;
} item;
};

int main() {
struct item s;
s.item.book.title = "...
s.item.book.title = "...
s.item.book.num_pages = ...
printf("%ld", sizeof(s));
}
//28

Note:- When we are storing book, size = 28 bytes
" " " " shirt, " = 28 bytes

Size = 8 + max(20, 16) = 8 + 20 = 28 bytes

Application of unions (Part-2)

Creating your own mixed type data structure
How good it is that if we have an array containing mixed type data

Eg) typedef union {
int a; char b; double c; } data;
int main() {
data arr[10];
arr[0].a = 10; arr[1].b = 'a'; arr[2].c =

So we have succeeded in creating an array containing mixed type data.

Why not Structures?

Assuming
size of (int) = 4 bytes
size of (char) = 1 byte
size of (double) = 8 bytes

Same structure & union of eg1 will take memory 13 & 8 bytes respectively.

So for arr[10] structure = 130 bytes
union = 80 ||

Enumeration (enum) in C

→ An enumerated type is a user defined type which is used to assign names to integral constants because names are easier to handle in program.

eg enum Bool {false, True};
int main() {
enum Bool var;
var = True;
printf("%d", var);
return 0; //1
}

false & True are names to integral constants

→ If we donot assign values to enum names then automatically compiler will assign values to them starting from 0
So false = 0, True = 1

Hey! But we can also use #define to assign name to integral constants. Then, why do we even need enum?

Need of Enumeration
1) Enums can be declared in local scope.
2) Enum names are automatically initialized by compiler.

Reason #1 eg

```
int main() {
    enum Bool{False, True} var;
    var = True;
    printf("%d", var);
    return 0;
}
```

this enum is not visible outside main().

Not possible with #define.

Reason #2 already covered.

Fact #1: Two or more names can have same value.

```
int main() {
    enum point{x=0, y=0, z=0};
    printf("%d %d %d", x, y, z); // 0 0 0
    return 0;
}
```

Scanned with CamScanner

Fact #2. We can anign values in any order. All unanigned names will get value as value of previous name +1;

eg int main() {
enum point { y=2, x=34, t, z=0 };
printf("%d %d %d %d", x, y, z, t); //34 2 0 35
}

Fact #3: Only integral values are allowed

int main() {
enum point {x=34, y=2.5, z=0};
printf("%d %d %d", x, y, z);
}

Error: Enumerator value for 'y' is not an integer constant.

Fact #4: All enum constants must be unique in their scope.

int main() {
enum point1 {x=34, y=2, z=0};
enum point2 {x=4, p=25, q=1};
printf("%d %d %d %d %d", x, y, z, p, q);
}

// Error: Redeclaration of enumerator 'x'

Scanned with CamScanner

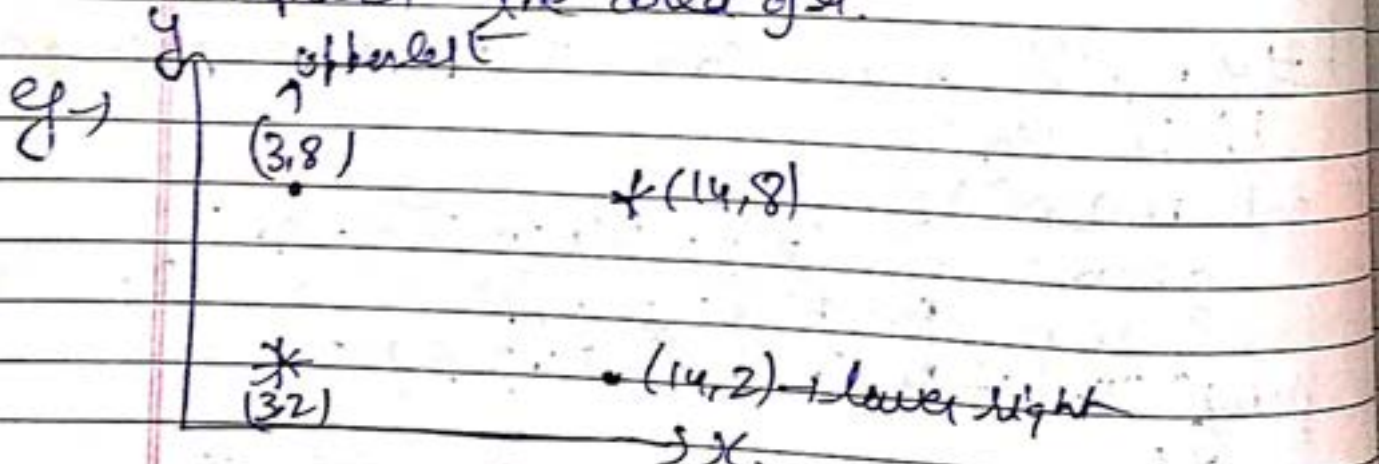
Calculating area of rectangle using structures.

Q-1 The following structures are designed to store information about objects on a graphics screen:

```
struct point { int x, y; };
struct rectangle { struct point upperleft, lowerright; };
```

A point structure stores the x & y coordinates of a point on the screen. A rectangle structure stores the coordinates of upper left & lower right corners of the rectangle.

Write a function that accepts rectangle structure r as an argument & computes the area of r.



14-3 length 8-2 breadth

So actually no need of * • are sufficient

Length = lower right x − upper left x;

Breadth = upper left y − lower right y;

Scanned with CamScanner

Date
Page 41

```
#include <stdio.h>
struct point {
    int x;
    int y;
};
struct rectangle {
    struct point upper_left;
    struct point lower_right;
};
int area(struct rectangle r) {
int length, breadth;
length = r.lower_right.x - r.upper_left.x;
breadth = r.upper_left.y - r.lower_right.y;
return length * breadth;
}
int main() {
    struct rectangle r;
printf("Enter the upper left coordinates of
the rectangle: \n");
scanf("%d %d", &r.upperleft.x, &r.upperleft.y);

printf("Enter lower right coordinates \n");
scanf("%d %d", &r.lowerright_x, &r.lower_right.y);

printf("Area of rectangle: %d", area(r));
return 0;
}
```

Scanned with CamScanner

Q. struct {
double a;
union {
char b[4];
double c;
} int d;
} e;
char f[4];
} s;
If char value occupy 1 byte, int values occupy 4 bytes, & double values occupy 8 bytes, how many space will a Compiler allocate for s?
(Assume that compiler leaves no holes in between);

Ans) 20 bytes

Q-1 union {
double a;
struct {
char b[4];
double c;
int d;
} e;
char f[4];
} u;

Ans:= 16 bytes

Scanned with CamScanner

Introduction to data structure.

What is data?

Dictionary definition: The quantities, characters or symbols on which operations are performed by a computer, which may be stored & transmitted in form of electrical signals and recorded on magnetic, optical or mechanical recording media.

eg $c = a + b;$
data

When data becomes information?

DATA: TEERPSAI SI EMAN YM
INFORMATON. MY NAME IS JASPREET.

If data is arranged in a systematic way then it gets a structure & becomes meaningful.

→ This meaningful or processed data is called information.

Its not difficult to understand that data needs to be managed in such a way so that it can produce some meaningful information.

To provide an appropriate way to structure data, we need to know about data structure.

Scanned with CamScanner

What is a data structure?
A data structure is the systematic way to organise data so that it can be used efficiently.

Eg: Arrays
Instead of creating multiple variables of same type why not create an array to store all the values.

Storing strings is equivalent of storing a sequence of characters. They requires an array.

Real life eg's.

1) Stack data structure is used in implementing redo & undo feature.

eg:- The primary goal of Neso Academy is to give quality education.

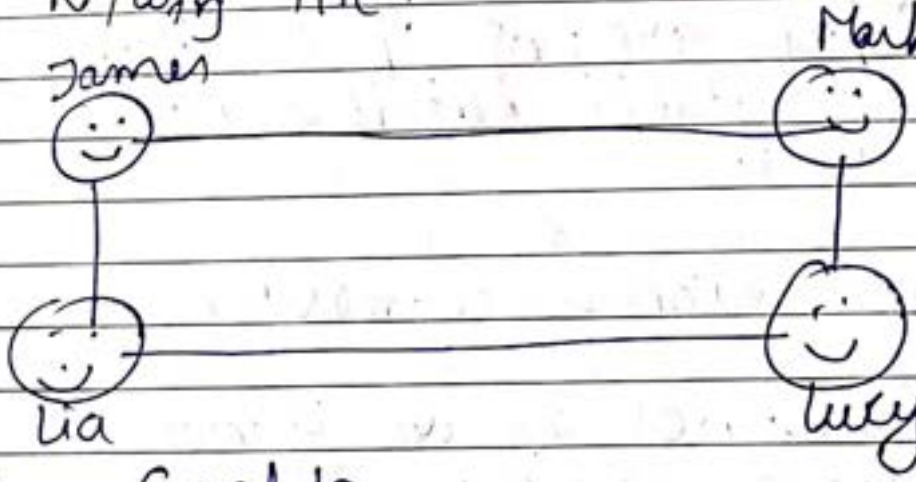


Scanned with CamScanner

Q2 → Which DS is used to store images as a bitmap?
Ans → Arrays. How?

Bitmap images are stored as a series of tiny dots called pixels.
Here, each pixel is actually a small square that is assigned color & then arranged in a pattern to form an image.

Q3 - Storing friendship information on a social N/W like FB?



Ans → Graph

Data types Vs Abstract Data types →

(What is a data type)
2 imp things about data types:
1) Defines a certain domain of values.
2) Defines operations allowed on those values.

Q int type
— Takes only integer values
— Operations: addition, subtraction, multiplication, bitwise operations etc.

Scanned with CamScanner

float type
→ Takes only floating point values
→ Operations: addition, subtraction, multiplication, division etc.
(bitwise & % operations are not allowed).

User defined data types
→ In contrast to primitive data types there is a concept of user defined data types.
→ The operations & values of user defined data types are not specified in the language itself but is specified by the user.

Eg. Structure, union & enumeration.

By using structures we are defining our own type by combining other types.

Eg struct point {
    int x;
    int y;
};

point is new data type having 2 int values

Scanned with CamScanner

Page 4.7

→ Abstract Data Type (ADT)

ADTs are like user defined data types which defines operations on values using functions without specifying what is there inside the function & how the operations are performed.

Eg: Stack ADT.

A stack consists of elements of same type arranged in a sequential order.

Operations
initialize() - initializing it to be empty
push() - Insert an element into stack
pop() - Delete an element from stack
isEmpty() - checks if stack is empty
isFull() - checks if stack is full

→ Think of ADT as a black box which hides the inner structure & design of the data type from the user.

→ There are multiple ways to implement an ADT.
Eg: A stack ADT can be implemented using arrays or linked lists.

Why ADT?
→ The program which uses data structure is called a client program
→ It has access to ADT i.e interface
→ The program which implements the data structure is known as implementation

Scanned with CamScanner

Advantage

→ Let say, if someone wants to use the stack in the program, he can simply use push & pop operations without knowing it's implementation.

→ Also, if in future, the implementation stack is changed from array to linked list then the client program will still work in same way without being affected.

Conclusion → ADT provides abstraction.

What is a data structure?

→ A data structure is the organization of the data in a way so that it can be used efficiently.

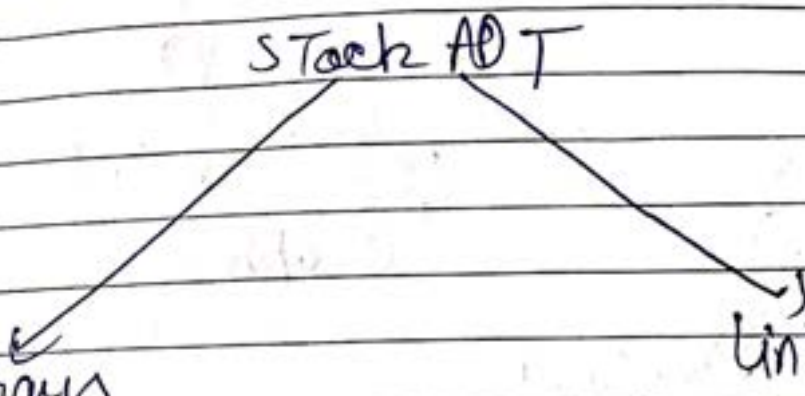
In other words, it is used to implement an ADT.

for eg - In order to implement stack ADT we can use an array data structure or linked list data structure.

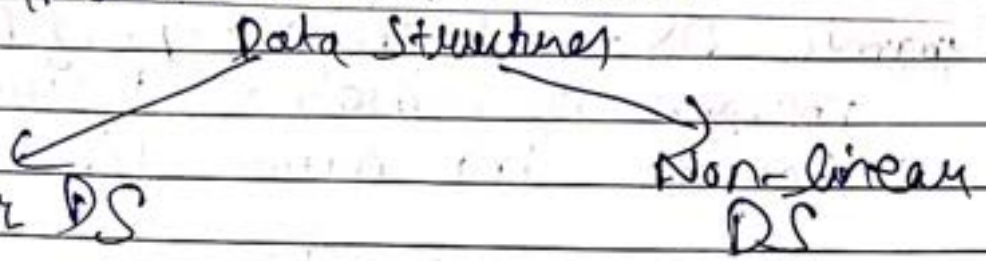
ADT tells us what is to be done & data structures tells us how to do it.

How to know which data structure to use for a particular ADT?

Which data structure?
In reality, different implementations are compared for time & space efficiency. The one best suited according to the current requirement of the user will be selected.



Advantages of data structures
1) Efficiency → proper choice of data structures make program efficient in terms of space & time
2) Reusability → one implementation can be used by multiple client programs.
3) Abstraction → Data structure is specified by an ADT which provides a level of abstraction. The client program doesn't have to worry about implementation details.



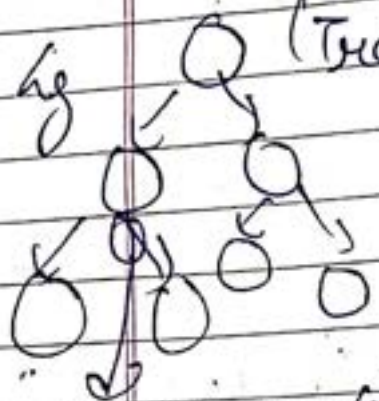
Linear DS → A DS is linear when all the elements are arranged in a linear (sequential) order
OR ADS in which each element has at most 1 predecessor and at most 1 successor
eg → Array, Stacks, Linked list, Queue

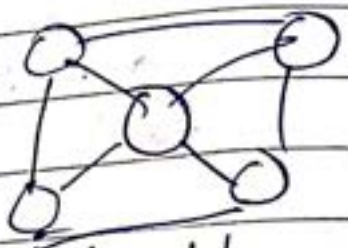
Scanned with CamScanner

Non-linear DS

→ A DS is non-linear when all elements are not arranged in a linear (sequential) order. There is no linear arrangement of elements.

eg (Tree)





Graph

2) Succession (Non-linear)

Given further classification

1) Static DS: In these type of DS, the memory is allocated at compile time. Therefore max size is fixed.

Advantage: Fast access

Disadvantage: Slower insertion & deletion.

eg Array

2) Dynamic DS → In these type of DSs, the memory is allocated at run time. Therefore maximum size is flexible

Advantage: Faster insertion & deletion

Disadvantage: Slower access

eg- Linked list

Scanned with CamScanner

Basics of Asymptotic Analysis [1]

Efficiency: Efficiency of DS is always measured in time & space. An ideal DS could be the one that takes the least possible time for all its operations & consumes the least memory space.

→ Our focus will be on finding time complexity.

Q On what basis, we could compare the time complexity of the DSs?

Ans Compare the DS on the basis of operations performed on them.

Let say 100 elements

Eg: arr [1|2|6|3|8|56|32|34]

Target: add data at beginning of the list

[1|2|6|3|8|56|32|34|] ← 100 elements

[4]

So very difficult task

Linked list

head → [1|2000] → [2|3000] → [6|NULL]
1000 3000

Add data at beginning

[1|2000] → [2|3000] → [6|NULL]
1000 2000 3000

head → [4|1000]
4000

So easy

Scanned with CamScanner

Conclusion → Inserting an element at the end of the list is way faster in DS1 than always.

- In above way, we can compare DSs, and this will ultimately give the idea to select the best possible data structure for a particular operation

But how to find time complexity or running time of an operation performed on DS1?

Method #1: Examine the exact running time on some machine & turn the timer on. Run the operation for different inputs on the data structure you want to compare one by one. and see how much time a particular operation will take on these data structures. The one which takes less time is best performer.

Problems with this approach: It might be possible that for some input size, first data structure gives best performance & for the other input size the second gives the best performance.

Scanned with CamScanner

→ It might also be possible that for some machine for some I/P size of data structure, it is performing well while in other machine the second data structure is performing well with a different size.

∴ Examining the exact running time is not the best solution to calculate the time complexity.

Points to be noted:

→ Measuring the actual running time is not practical at all

→ The running time generally depends on the size of the input.

eg arr [1] [2] [3]

but what if we have 10000 elements in an array?

In that case, we have to make 10000 shifts which will take 10000 units of time. This clearly shows that the running time generally depends on size of input.

∴ If the size of I/P is n, then f(n) is a function of n denotes the time complexity

In other words f(n) represents the number of instructions executed for the input value n.

Scanned with CamScanner

for eg:-

```
for (i=0; i<n; ++i) {
    printf("Helo World");   ← Assuming this instruction will take 1 unit of time
}
printf("Instruction is excuted n number of times.");
```

∴ $f(n) = n$.

But, how to find $f(N)$?

→ We can compare 2 DS for a particular operation by comparing their $f(n)$ value.

→ We are interested in growth rate of $f(n)$ wrt n because it might be possible that for smaller I/P size, one DS may seem better than the other but for larger I/P size it may not.

→ This concept is applicable in comparing the 2 algorithms as well.

Eg $f(n) = 5n^2 + 6n + 12$

$n = 1$ % of running time due to $5n^2 = \frac{5}{5+6+12} \times 100$

$= 21.74\%$

% of running time due to $6n = \frac{6}{5+6+12} \times 100 = 26.08\%$

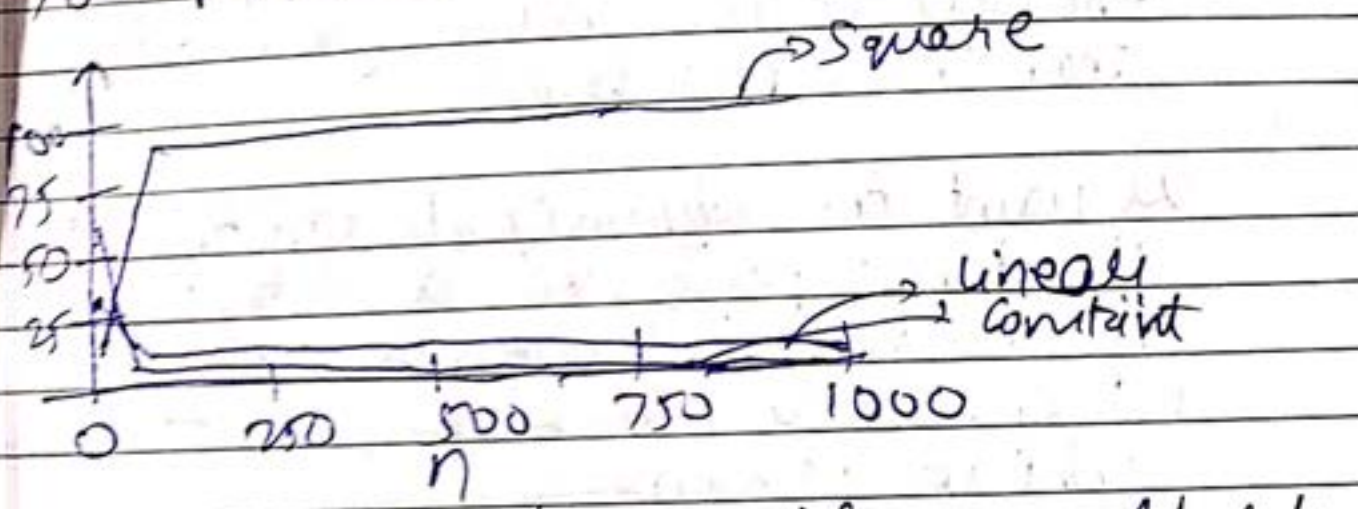
n
1
10
100
100

Scanned with CamScanner

% of running time due to 12 $= \frac{12}{5+6+12} \times 100 = 52.17\%$

n	$5n^2$	$6n$	12
1	21.74%	26.09%	52.17%
10	87.41%	10.49%	2.09%
100	99.79%	1.19%	0.02%
1000	99.88%	0.12%	0.0002%

} we have to see the growth rate



So there is no harm if we can eliminate the rest of the terms as they are not contributing much to the time.

So $f(n) = 5n^2$

We are getting the approximate time complexity & we are satisfied with this result because this approximate result is very near to the actual result.

This approximate measure of time complexity is called Asymptotic complexity.

Scanned with CamScanner

Basics of Asymptotic Analysis

Big O Notation → used to measure the performance of any algorithm by providing the order of growth of the function.

It gives the upper bound to a function by which we can make sure that the function will never grow faster than this upper bound.

We want the approximate runtime of the operations performed in data structure we are not interested in exact runtime Big O notation will help us to achieve the same.

Defn → If $f(n)$ & $g(n)$ are 2 functions, then $f(n) = O(g(n))$ if there exists constants c & n_0 such that $f(n) \leq c \cdot g(n)$, for all $n \geq n_0$

→ This simply means that $f(n)$ does not grow faster than $g(n)$.

Eg: i) $f(n) = n$ $g(n) = 2n$

Is $f(n) = O(g(n))$?

$f(n) \leq c \cdot g(n)$

$n \leq c \cdot 2n$ for $c = 1$

& $n_0 = 1$ ✓

Scanned with CamScanner

2) $f(n) = 4n + 3$, $g(n) = 5n$

Is $f(n) = O(g(n))$?

$f(n) \leq c \cdot g(n)$

$4n + 3 \leq c \cdot n$

Take $c = 5$

$5n - 4n \geq 3$

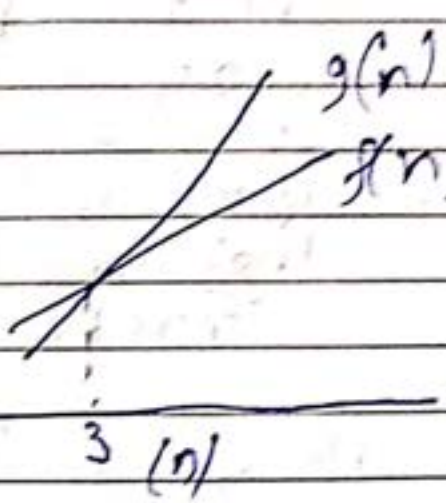
$n \geq 3$

$\therefore f(n) \leq c \cdot g(n)$ for all $n \geq 3$ where $c = 5$ & $n_0 = 3$

(Yes)

Explanation

n	f(n)	g(n)
1	7	5
2	11	10
3	15	15
4	19	20
5	23	25
6	27	30
7	31	35
8	35	40
9	39	45



$f(n) = O(g(n))$ simply tells the worst case time complexity of $f(n)$ i.e $5n$ or linear ie In future $f(n)$ can't be quadratic or cubic or exponential etc.

Scanned with CamScanner

- We are always interested in finding the growth rate of the function.

→ For this purpose, we plug in different values of N into the function & plot a graph to see how it grows.

But why do we need to do this?

→ Let say, we are comparing 2 data structures for a particular operation. It might be possible that for some set of inputs, first data structure may give the best results while for some other set of inputs, second data structure may give the best results.
We can't simply predict that after which value of N, we can be sure that one DS will perform better than other DS.

- For eg n=3 in previous case.

→ This is the reason why we are plugging in different values of N and are interested in the growth rate of the functions representing the time taken by the operations performed or data structure.

- But isn't it a tedious task to plug different values of N & find growth rate of the function for comparison?

Here
Big O
growth
plug

And
give
und
for

→ That
al

Eg

We

Be
I

Scanned with CamScanner

Here comes Big O Notation-)

Page 59

Big O Notation helps us in finding the growth rate of the function without plugging dif values of N.

And the best part is, it gives the least upper bound on the function which gives the surety that the function under consideration will never grow faster than this upper bound.

→ That least upper bound tells how worst an algorithm can perform.

Eg $f(n) = 5n^2 + 4$ $g(n) = n^2$

Is $f(n) = O(g(n))$?

Let $c = 6$

$5n^2 + 4 \leq 6n^2$

$6n^2 \geq 5n^2 + 4$

$n \geq 2$

for n $n \geq 2$

where $c = 6$ and $n_0 = 2$

$f(n) = O(n^2)$

→ Big O notation is simply saying the task. It eliminates all the unnecessary terms from the function which are not contributing much in overall running time Eg → 4 in above eg

Scanned with CamScanner

Growth rate of standard functions

$\log n < n < n \log n < n^2 < n^3 < 2^n$

Program to find sum of first N Natural numbers.

```
int main(){
int i, sum=0, n;        <- 1 time
scanf("%d", &n);        <- 1 time

for(i=1; i<=n; i++){
  sum += i;             <- n times
}
printf("%d", sum);      <- 1 time
return 0;               <- 1 time
}
}
```

$n+4$ times $\simeq n$ (Simply eliminate k)

growth rate

So $g(n) = n$

$f(n) = n+4$

$n+4 \le 2 \cdot n$

$n \ge 4$ $\quad f(n) \le c \cdot g(n)$

$\therefore f(n) = O(g(n))$ for all $n \ge n_0$

$c = 2, n_0 = 4$

$f(n) = O(n)$ → Linear growth

→ So alter

```
int main() {
int i, sum=0, n;
scanf("%d", &n);
printf("%d", n*(n+1)/2);
return 0;
}
```

$f(n) = 4$

let $g(n) = 1$

Is $f(n) = O(g(n))$

$f(n) \leq c \cdot g(n)$

$4 \leq c \cdot 1$

Take $c = 5$ $4 \leq 5$

So $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$ where $c = 5$, $n_0 = 1$

$f(n) = O(1)$ Constant growth

Guidelines for Asymptotic Analysis

1) Loops

```
for (i=1; i<=n; i++)        // n times
    // 1 statement
```

So $O(n)$

2) Nested Loops

```
for (i=1; i<=n; i++) {      outer n
    for (j=1; j<=n; j++)    inner n
        // 1 statement      total n x n = n^2
}
```

Scanned with CamScanner

3) Consecutive statements

a)
```
int x=2;
int i;
x=x+1;
```
} 3 units
O(1)

b)
```
int x=2;
int i;
x=x+1;
```
} 3

```
for(i=1; i<=n; i++){
    // statement
}
```
n

```
for(i=1; i<=n; i++){
    for(j=1; j<=n; j++){
        // n^2
    }
}
```

time = n^2+n+3 = $O(n^2)$

do by taking $f(n)=n^2+n+3$
$g(n)=n^2$

4) If-then-else statements

```
scanf("%d", &n);
if(n==0){
    //statement
}else{
for(i=1; i<=n; i++){
```

i.e condition
↑
Test part + If part or else part (which ever is longest)

Here statement inside if takes constant time

So if = 1+1 = O(1)

else = 1 + O(n) = O(n)

Running time of condition must be calculated in else part.

Logarithmic Complexity

$\log_2(8) = 3$ This says "How many times, 2 has been multiplied by itself in order to obtain value 8."

Time Complexity → logarithmic time complexity is achieved when the problem size is cut down by a fraction

```
for [i=1; i<=n; ) {
    //statement
    i = i * 2;
}
```

Iteration			
1	initially	i = 1	2^0
2		i = 2	2^1
3		i = 4	2^2
4		i = 8	2^3
k		i = n	2^{k-1}

Scanned with CamScanner

$n = 2^{k-1}$

$\log_2 n = k-1$

$k = \log_2 n + 1 = O(\log_2 n)$

Eg: for (i=1; i<=32;) {
printf("Hello");
i = i*2;
}

= 6 iterations

Eg 2 for (i=n; i>=1;) {
i = i/2;
}

Iter	i
1	$i = \frac{n}{2^0}$
2	$i = \frac{n}{2^1}$
3	$i = \frac{n}{2^2}$
4	$i = \frac{n}{2^3}$
⋮	⋮
k	$i = \frac{n}{2^{k-1}} + 1$

$\frac{n}{2^{k-1}} = 1$, $n = 2^{k-1}$

$k = \log_2 n + 1$

$= O(\log_2 n)$

Scanned with CamScanner

Q. Find time complexity of

```
void fun(int n) {
(1) - int i, j, k, count=0;
(2) - for (i=n/2; i<=n; i++)
(3) -   for (j=1; j+n/2<=n; j++)
(4) -     for (k=1; k<=n; k=k+2)
            count++;
}
```

Ans (1) — O(1)

(2) — Iter 1 $i = \frac{n}{2} + 0$

2 $i = \frac{n}{2} + 1$

$+2$

Iter k $i = \frac{n}{2} + k - 1 = n$

$O(n)$ ($\because \frac{n}{2} + k - 1 = n$, $k = \frac{n}{2} + 1$)

(3) — Iter 1 $j = 1$

2 $j = 2$

⋮

k $j = k = \frac{n}{2}$, $O(n)$

(4) $O(\log_2 n)$

Ans = $O(n^2 \log_2 n)$

Scanned with CamScanner

Q- Time complexity of
void fun(int n){
if(n<=1) return;
int i, j;
for(i=1; i<=n; i++)
for(j=1; j<=n; j++){
printf("Hello\n");
break;
}
}

Ans O(n) because 2nd for loop has break statement

Q- Time complexity of
void fun(int n){
① — int i, j;
② — for(i=1; i<=n/3; i++)
③ — for(j=1; j<=n; j+=4)
printf("Hello\n");
}

Ans → ① - O(1)
② - O(N)
③ - O(N) Total O(N²)
You can derive also

Understanding Void Pointers

→ Void pointer is a pointer which has no associated data type with it.

Scanned with CamScanner

It can point to any data of any data type and can be typecasted to any type.

Eg
```
int main(){
    int n=10;
    void *ptr = &n;
    printf("%d", *(int*)ptr); //10
    return 0;
}
```

Note- We can't dereference a void pointer directly, we have to first typecast it.

Use of void pointer → malloc & calloc function returns a void pointer. Due to this reason, they can allocate a memory for any type of data.

Syntax `void* malloc(size_t size);`

malloc & calloc use to allocate memory at runtime.

Understanding null pointer → A NULL pointer is a pointer that does not point to any memory location. It represents an invalid memory location. When a NULL value is assigned to a pointer, then the pointer is considered as NULL pointer

Eg
```
int main(){
    int *ptr = NULL;   <- NULL pointer
    return 0;
}
```

Scanned with CamScanner

Uses of NULL pointer

(1) It is used to initialize a pointer when that pointer isn't assigned any valid memory address yet.

Eg int main() {
    int *ptr = NULL; // It is good practice to initialize pointer with NULL whenever it has not assigned any
    return 0;
}

(2) Useful for handling errors when using malloc function.

Eg: int main() {
    int *ptr;
    ptr = (int *)malloc(2*sizeof(int)); // malloc return NULL if memory not allocated
    if (ptr == NULL)
        printf("Memory could not be allocated");
    else
        printf("Memory allocated successfully");
    return 0;
}

Facts about NULL Pointer

(1) The value of NULL is 0. We can either use NULL or 0 but this 0 is with in context of pointers and is not equivalent to integer 0.

Eg int main() {
    int *ptr = NULL;
    printf("%d", ptr); // O/P 0
    return 0;
}

Scanned with CamScanner

Side note - Size of normal pointer = 8 bytes / Device dependent

2. Size of NULL pointer depends upon the platform & is similar to the size of normal pointer.

```
Eg int main(){
   printf("%d", sizeof(NULL)); // 8
   return 0;
   }
```

Best practices
→ It is a good practice to initialize a pointer as NULL
→ It is a good practice to perform NULL check before dereferencing any pointer to avoid surprises.

Understanding Dangling pointers → A dangling pointer is a pointer which points to some non-existing memory location.

```
Eg 1 int main(){
     int *ptr = (int *)malloc(sizeof(int));
     ---
     ---
     free(ptr)   // Memory is now released
     return 0;      deallocate the memory
   }                allocated by malloc
```

But the pointer is still pointing to the deallocated memory, because we have not reinitialized the memory

Scanned with CamScanner

Solution

```
int main() {
  int *ptr = (int *) malloc (sizeof (int));
  ...
  ...
  free (ptr);
  ptr = NULL;   //Now ptr is no more dangling
  return 0;
}
```

Eg 2

```
#include <stdio.h>
int *fun() {
    int num = 10;
    return &num;
}
int main() {
    int *ptr = NULL;
    ptr = fun();
    printf("%d", *ptr); //segmentation fault
    return 0;
}
```

C

because ptr points to num which is local to fun() & it becomes dangling

Understanding Wild Pointers

→ Wild pointers are also known as Uninitialized pointers.

→ These pointers usually point to some arbitrary memory location & may cause a program to crash or misbehave

eg int main(){
int *p;
*p=10; //crash/Segmentation fault/Undefined behaviour;
return;
}

How to avoid wild pointers?

1) Initialize them with the address of a known variable. eg → int main(){ int var=10; int *p; p=&var; }

2) Explicitly allocate the memory & put the values in the allocated memory.

eg: int main(){
int *p= (int *)malloc(sizeof(int));
*p=20; //Allowed
free(p);
return 0;
}

Static Memory Allocation → Memory allocated during compile time is called static memory. The memory allocated is fixed & cannot be increased or decreased during run time.

eg
int main(){
int arr[5]={1,2,3,4,5};
}

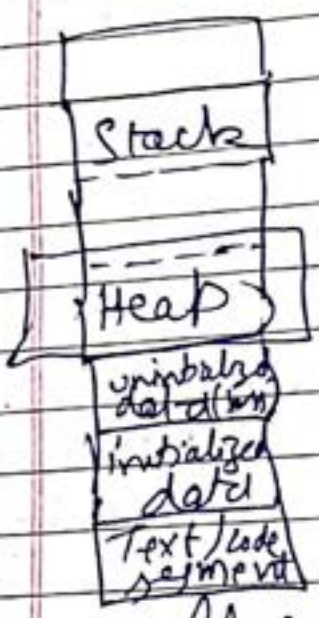
Problems faced in static memory allocation
If you are allocating memory for an array during compile time then you have to fix the size. at the time of declaration. Size is fixed & user cannot change the size of array at run time.

Scanned with CamScanner

→ If the values stored by the user in the array at runtime is less than the size specified then there will be wastage of memory.

→ If the values stored by the user in the array at runtime is more than the size specified then the program may crash or misbehave.

Dynamic Memory Allocation → The process of allocating memory at the time of execution is called dynamic memory allocation.



Heap is segment of memory where dynamic memory allocation takes place

Unlike stack where memory is allocated or deallocated in a defined order, heap is an area of memory where memory is allocated or deallocated without any order or randomly.

There are certain built-in functions that can help in allocating or deallocating some memory space at runtime

Please Note: Pointers play an important role in dynamic memory allocation.
Allocated memory can only be accessed through pointers

Built-in functions =)
1) malloc()
2) calloc()
3) realloc()
4) free()

Malloc() → Malloc is a built in function declared in the header file <stdlib.h>

malloc is short name for "memory allocation" and is used to dynamically allocate a single large block of continuous memory according to the size specified.

Syntax (void*) malloc(size_t size)
malloc function simply allocates a memory block according to the size specified in the heap & on success it returns a pointer pointing to the first byte of the allocated memory else returns NULL. void pointer

size_t is defined in <stdlib.h> as unsigned int



Scanned with CamScanner

Why void pointer?

→ malloc doesn't have an idea of what it is pointing to.

→ it merely allocates memory requested by user without knowing the type of data to be stored inside the memory.

→ The void pointer can be typecasted to an appropriate type.

eg int *ptr = (int *) malloc(4);

malloc allocates 4 bytes of memory in heap & the address of the first byte is stored in the pointer ptr.

eg prog → #include<stdio.h>
#include<stdlib.h>
int main(){
int i, n;
printf("Enter the # of integers: ");
scanf("%d", &n);
int *ptr = (int *)malloc(n*sizeof(int));
if (ptr == NULL){
printf("Memory not available.");
exit(1); //exit out of program
}
for (i=0; i<n; i++){
printf("Enter an integer: ");
scanf("%d", ptr+i);
}

Scanned with CamScanner

```
for(i=0; i<n; i++)
    printf("%d", *(ptr+i));
return 0;
}
```

calloc(): calloc() function is used to dynamically allocate multiple blocks of memory. It is different from malloc in 2 ways:

1) calloc() needs 2 arguements instead of just 1.
Syntax: void *calloc(size_t n, size_t size);
(n ↑ # of blocks; size ↑ size of each block.)

Eg: int *ptr = (int *) calloc(10, sizeof(int));
|||
int *ptr = (int *) malloc(10 * sizeof(int));

2) Memory allocated by calloc is initialized to zero. It is not the case with malloc (garbage value);

Note: malloc & calloc both return null when sufficient memory is not available in the heap.

calloc stands for clear allocation
malloc " " memory "

Scanned with CamScanner

realloc() → reallocation
(stdlib)

realloc() function is used to change the size of the memory block without losing the data.

Syntax: void *realloc(void *ptr, size_t new_size);
ptr → Pointer to previously allocated memory

→ ON failure, realloc returns NULL

Eg int *ptr = (int *) malloc (sizeof(int));
ptr = (int *) realloc (ptr, 2*sizeof(int));

→ This will allocate memory space of 2 ints
→ Also, this function moves the contents of the old block to a new block & the data of the old block is not lost.
→ We may lose the data when the new size is smaller than the old size
→ Newly allocated bytes are uninitialized

Eg
```
#include <stdio.h>
#include <stdlib.h>
int main() {
  int i;
  int *ptr = (int *) malloc (2*sizeof(int));
  if (ptr == NULL) {
    printf ("Memory not available.");
    exit(1);
  }
```

Scanned with CamScanner

```
Rintf("Enter 2 numbers: \n");
for(i=0; i<2; i++){
    scanf("%d", ptr+i);
}

ptr=(int *)realloc(ptr, 4*sizeof(int));
if(ptr==NULL){
    printf("memory not available!");
    exit(1);
}
printf("Enter 2 more integers: \n");
for(i=2; i<4; i++)
    scanf("%d", ptr+i);

for(i=0; i<4; i++)
    printf("%d", ptr+i);
}
```

free() → used to release the dynamically allocated memory in heap.

Syntax: void free(ptr)

The memory allocated in heap will not be released automatically after using the memory. The space remains there & can't be used

It is programmer's responsibility to release the memory after used

Scanned with CamScanner

Eg-

```
int main(){
int *ptr = (int*) malloc(4*sizeof(
---
---
free(ptr); ptr=NULL;
}
```

good practice

code

```
#include<stdio.h>
#include<stdlib.h>
int *input(){
 int *ptr, i;
 ptr = (int*)malloc(5*sizeof(int));
printf("Enter 5 numbers: ");
for(i=0; i<5; ++i)
    scanf("%d", ptr+i);
return ptr;
}
int main(){
 int i, sum=0;
 int *ptr = input();
 for(i=0; i<5; i++)
    sum += *(ptr+i);
printf("Sum is: %d", sum);
free(ptr);
ptr = NULL;
return 0;
}
```

Scanned with CamScanner

Structures & functions

Date / Page 79

Topics

1) Passing structure member as argument
2) Passing structure variable as ''
3) Passing pointers to structures as ''
4) Returning as structure variable from the function
5) Returning a pointer to a structure from the function
6) Passing array of structures as argument.
7) Self referential structures.

1) Structure members as argument → Just like variables, we can pass structure members as arguments to a function.

```
struct student{
    char name[50];
    int age;
    int roll_no;
    float marks; };
void print (char name[], int age, int roll, float marks)
{ printf("%s %d %d %.2f", name, age, roll, marks);
int main(){
    struct student s1 = {"Nick", 16, 50, 72.5};
    print (s1.name, s1.age, s1.roll_no,
           s1.marks);
}
```

Scanned with CamScanner

Call by reference

Instead of passing the copies of the structure members, we can pass their addresses (or references)

eg
```
struct charset {
    char s;
    int i;
};
void keyValue(char *s, int *i) {
    scanf("%c %d", s, i);
}
int main() {
    int j;
    struct charset cs;
    keyValue(&cs.s, &cs.i);
    printf("%c %d", cs.s, cs.i);
    return 0;
}
```

→ no need to put bracket because '.' has > precedence than &

2) Structure variable as an argument

Instead of passing structure members individually, it's a good practice to pass a structure variable as an argument.

Unlike arrays, name of the structure variables are not pointers.

It's a pass by value

eg
```
#include <stdio.h>
struct point {
    int x;
    int y;
};
void print (struct point p) {
    printf ("%d %d \n", p.x, p.y);
}
int main () {
    struct point p1 = {23, 45};
    struct point p2 = {56, 90};
    print (p1);
    print (p2);
    return 0;
}
```

O/P
23 45
56 90

3) Pointer to structure as an argument
If the size of the structure is very large then passing the copy of whole structure is not efficient.
Better Choice: Pass the address of the structure.

→ Use the arrow operator (->) to access the structure members inside the called function

eg
```
#include <stdio.h>
struct point { int x; int y; };
void print (struct point *ptr) {
    printf ("%d %d \n", ptr->x, ptr->y);
}
int main () {
    struct point p1 = {23, 45}; struct point p2 = {56, 90};
    print (&p1); print (&p2); return 0;
}
```

Scanned with CamScanner

Returning structure variable from function

→ Returning a structure variable from a function is similar to returning variable from a function.

```
#include<stdio.h>
struct point {
    int x;
    int y;
};
struct point edit(struct point p){
    (p.x)++;
    p.y = p.y+5;
    return p;
}
void print(struct point p){
    printf("%d %d\n", p.x, p.y);
}
int main(){
    struct point p1={25, 45};
    struct point p2={56, 90};
    p1=edit(p1);
    p2=edit(p2);
    print(p1);
    print(p2);
    return 0;
}

//26 50
//57 95
```

Scanned with CamScanner

5) Returning a pointer to a structure from the function.

```
#include<stdio.h>
#include<stdlib.h>
struct point {
    int x;
    int y;
};
struct point *fun(int a, int b) {
    struct point *ptr = (struct point *)malloc(1*sizeof(struct point));
    ptr->x = a;
    ptr->y = b+5;
    return ptr;
}
void print(struct point *ptr) {
    printf("%d %d\n", ptr->x, ptr->y);
}
int main() {
    struct point *ptr1, *ptr2;
    ptr1 = fun(2,3);
    ptr2 = fun(6,9);
    print(ptr1); print(ptr2);
    free(ptr1);
    free(ptr2);
    return 0;
}
```

// 2 8
// 6 14

Scanned with CamScanner

18/9/20

Passing array of structure as arguement

eg struct abc {
int a;
char b; };
struct abc arr[2] = {{1, 'A'}, {2, 'B'}};

Compiler will allocate contiguous block of memory for the data members of the structure

Code - #include <stdio.h>
struct point {
int x;
int y;
};
void print (struct point arr[]) {
int i;
for (i=0; i<2; i++)
printf ("%d %d \n", arr[i].x, arr[i].y);
}
int main () {
struct point arr[2] = {{1,2}, {3,4}};
print (arr);
return 0;
}

Introduction to Linked List

Date / Page 85

Task: Arrange the name of students according to first letter of their names

In memory

- Arrays
- Linked list

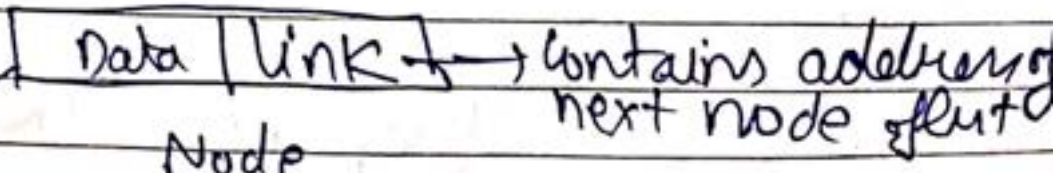
Arrays have some limitations & disadvantages we will see how linked list overcomes the limitation of arrays in subsequent lectures

Types of linked list

1) Single linked list: Navigation is forward only
2) Doubly " " : forward & backward navigation is possible
3) Circular linked list: Last element is linked to the first element

Single linked list

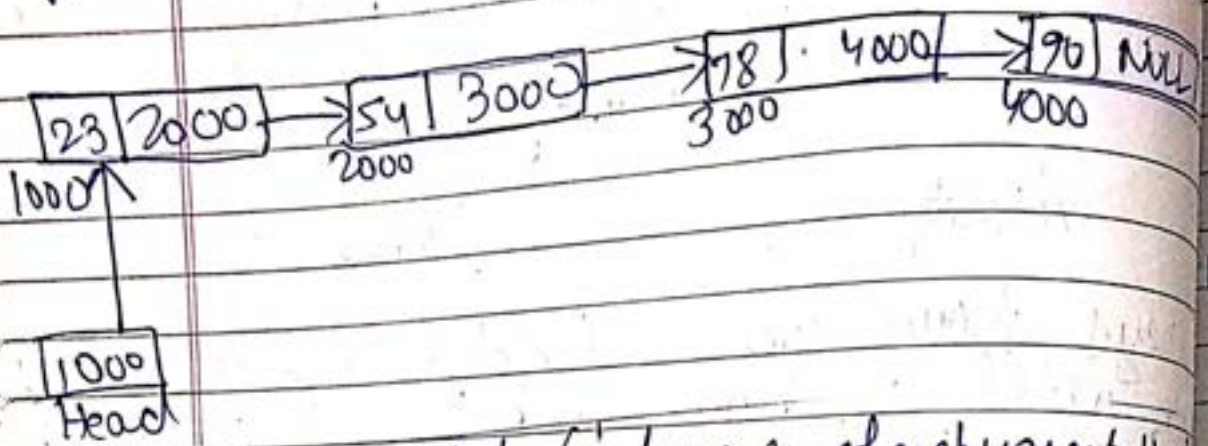
- A " " : " is a list made up of nodes that consists of 2 parts.
  a) data
  b) link



Scanned with CamScanner

Representation of a single linked list

Suppose we want to store a list of #s - 23, 54, 78, 90



Array V/s linked list (in terms of representation)

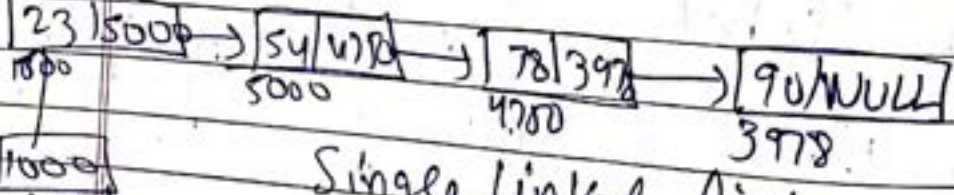
We want to store a list of #s : 23, 54, 78, 90

ARRAY

23	54	78	90
1000	1004	1008	1012

In array, elements are stored in consecutive memory locations.

So Array is sequential representation of list



Single linked list

→ Nodes are scattered here & there in the memory but they are still connected with each other.

→ Linked list is linked representation (not sequential) of list

Scanned with CamScanner

Creating a node of single linked list

We have missed self referential structures, let's get back

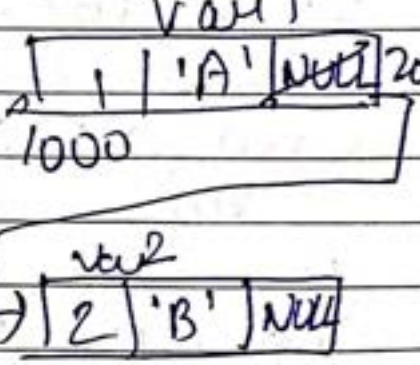
Self referential structures

Def: Those structures in which one or more pointers point to the structure of the same type

```
struct self{
    int p;
    struct self *ptr;
};
```

Eg

```
struct node{
    int i;
    char c;
    struct node *ptr;
};
int main(){
struct node var1,var2;
var1.i = 65;
var2.c = 'A';
var1.ptr = NULL;
var2.i = 66;
var2.c = 'B';
var2.ptr = 'NULL'
var1.ptr = &var2;
printf("%d %c", var1.ptr->i, var1.ptr->c);
}
```

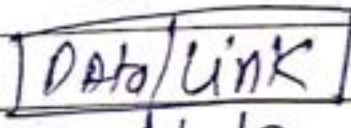


Scanned with CamScanner

Creating a node of single linked list

→ We will use self referential structure for creating a node of single linked list

```
struct node{
  int data;
  struct node *link;
};
```



In general

```
struct node{
  data_type member1;
  data_type member2;
  struct node *link;
};
```

multiple data part

Only 1 pointer (Singly linked list)

Creating a node in C

```
#include<stdio.h>
#include<stdlib.h>
struct node{
  int data;
  struct node *link;
};
int main(){
  struct node *head=NULL;
  head=(struct node *) malloc(sizeof(struct node));
//ifs head->data=45
  head->link=NULL; printf("%d",head->data);
```

because void part

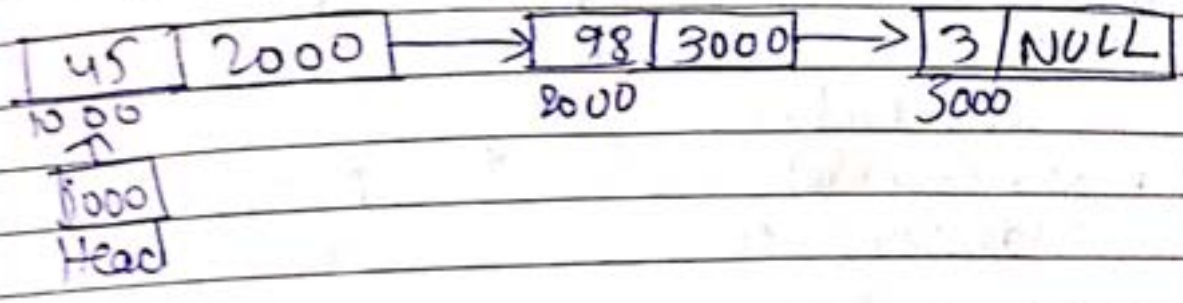
not essential but good coding practice

Scanned with CamScanner

Creating a single linked list

Page 89

Let's say we want to create




```
Code #include <stdio.h>
     #include <stdlib.h>
     struct node {
         int data;
         struct data *link;
     };
     int main(){
       struct node *head = (struct node*) malloc
                           (sizeof(struct node));
       head->data = 45;
       head->link = NULL;
     struct node *current = malloc(sizeof(struct node));
     current -> data = 98;
     current -> link = NULL;
     head -> link = current;

     // 1st way to add 3rd node
     struct node *current2 = malloc(sizeof(struct node));
     current2->data = 3;   current 2 -> link = NULL;
     current -> link = current2;
     //disadvantage: unnecessary pointer created
     //2nd way  head->link->link -> 3000  head->data->link
                                          =NULL
     Current = malloc (sizeof(struct node));
     Current -> data = 3;
     Current -> link = NULL;
     head -> link -> link = Current;
```

Scanned with CamScanner

Traversing a single linked list (counting nodes)

→ Traversing means visiting each node of a single linked list until the end node is reached

Our job is to calculate total # of nodes by traversing the linked list.

```
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *ln(c;
};
int main(){    write code yourself
 count_of_nodes(head);
}
void count_of_nodes(struct node *head){
int count=0;
if (head == NULL)
    printf("Linked list is empty");
struct node *ptr = NULL;
ptr = head;
while (ptr != NULL){
    count++;
    ptr = ptr -> link;
}
printf("%d", count);
}
```

Scanned with CamScanner

Traversing the linked list (Printing the data)

```
#include<stdio.h>
#include<stdlib.h>
struct node {
    int data;
    struct node *link;
};
int main(){
    print_data(head);
}
void print_data(struct node *head){
    if(head==NULL)
    printf("Linked list is empty");
struct node *ptr=NULL;
ptr= head;
while(ptr!=NULL){
printf("%d", ptr->data);
    ptr=ptr->link;
}
}
```

Traversing the list (Array v/s Linked list)

1) From while loops of linked list it is clear that time complexity: O(n) for both counting & printing the elements

2) In array
for counting — O(1)
for printing — O(n)

∴ size of (arr) / size of (int)

Scanned with CamScanner

Insertion at end of linked list

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *link;
};
int main(){
    add_at_end(head, 67);
}
void add_at_end(struct node *head, int dat
struct node *ptr, *temp;
ptr=head;
temp=(struct node*) malloc(sizeof(struct nod
temp->data=data;
temp->link=NULL;
while (ptr->link!=NULL)
  ptr=ptr->li
ptr->link=temp;
}
```

Insertion at end (linked list Vs array)

In above code while loop = O(n) = linked list

But, we have a better code, so let's see that code

Scanned with CamScanner

```
#include <stdio.h>
struct node* add
#include <stdlib.h>
struct node{
int data;
struct node *link; };

int main(){
struct node *head = malloc(size of (struct node));
head -> data = 45;
head -> link = NULL;
struct node *ptr = head;

ptr = add_at_end(ptr, 98);
ptr = ; ; ; ; ; (ptr, 3);
; ; ; ; ; ; ; (ptr, 67);
ptr = head;
while(ptr != NULL){
printf("%d", ptr->data);
ptr = ptr -> link;
}}
struct node *add_at_end(struct nde *ptr, int da
{ struct node *temp = malloc(size of (struct node));
temp -> data = data;
temp -> link = NULL;
ptr -> link = temp;
return temp;
}
```

Time complexity : O(1)

array - F.I.O

Insertion at end: Array

Array

Array not full | Array is full

Case 1: Array not full

```
int main(){
  int a[10];
  int i, n, freePos;
  printf("enter the # of elements:");
  scanf("%d", &n);

  for(i=0; i<n; i++)
    scanf("%d", &a[i]);
  freePos = n;
  freePos = add_at_end(a, freePos, 65);

  for(i=0; i<freePos; i++)
    printf("%d", a[i]);
}

int add_at_end(int a[], int freePos, int data){
  a[freePos] = data;
  freePos++;
  return freePos;
}
```

Time Complexity: O(1)

Scanned with CamScanner

Case 2: Array is full

```
int main() {
    int i, n, freePos;
    printf("Enter the # of elements: ");
    scanf("%d", &n);

    for(i=0; i<n; i++)
        scanf("%d", &a[i]);

    int size = sizeof(a)/sizeof(a[0]);
    freePos = n;
    if(n==size) {
        int b[size+2]; // b[size+1] also fine
        freePos = add_at_end(a, b, freePos, b[5]);
        for(i=0; i<freePos; i++)
            printf("%d", b[i]);
    }
}

int add_at_end(int a[], int b[], int n, int freePos, int data) {
    int i;
    for(i=0; i<n; i++)
        b[i] = a[i];
    b[freePos] = data;
    freePos++;
    return freePos;
}
```

Time complexity O(N)

Scanned with CamScanner

Summary

Linked list	Array
with traversal : O(n)	when array is full : O(n)
without " : O(1)	when " " not " : O(1)

LL has advantage : we can't guarantee whether array is full or not. but traversal can be always prevented.

Inserting Node at beginning of linked list

Code

```
#include<stdio.h>
#include<stdlib.h>
struct node {
    int data;
    struct node *link;
};
int main(){
struct node *head=malloc(sizeof(struct node));
head->data = 45;
head->link = NULL;

struct node *head = malloc(sizeof(struct node));
head -> data = 45;
head -> link = NULL;

struct node *ptr = malloc(sizeof(struct node));
ptr -> data = 98;
ptr -> link = NULL;
head -> link = ptr;
```

Scanned with CamScanner

```
int data = 3;
head = add_beg(head, data); -①
ptr = head;
while (ptr != NULL) {
printf("%d", ptr->data);
  ptr = ptr->link;
}
}
```

```
struct node* add_beg(struct node* head, int d)
{ struct node *ptr = malloc(sizeof(struct node));
  ptr -> data = d;
  ptr -> link = NULL;

  ptr -> link = head;
  head = ptr;
  return head;
}
```

Time complexity : O(1)

Common mistakes 1) In ① head is passed by value not reference so we are using head = add_beg(head, data);

2) ∵ head is like a variable of struct node type If it would have been by reference then code is add_beg(head, data); - simply

2) So for that reason we have to return head pointer from add_beg function. Is there a simple way?

Ans- Yes

Scanned with CamScanner

Inserting data at beginning of LL 2.0

~~#include~~ <

Change line 1 as

add_beg (&head, data);

& function as

void add_beg (struct node **head, int d){

struct node *ptr = malloc(sizeof(struct node));

ptr -> data = d;

ptr -> link = NULL;

ptr -> link = *head;

*head = ptr;

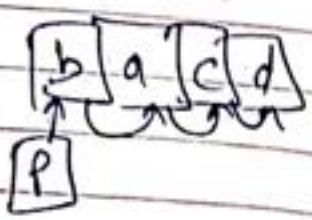
}

Time Complexity O(1)

Inserting data at beginning (LL V/s Array)

Linked list - O(1)

Array →



If there are n elements, n-shifts are required. So Time Complexity = O(n)

Code: #include main(){

int arr[10], data=10, i, n;

printf("Enter the # of elements:");

scanf("%d", &n);

Scanned with CamScanner

```
printf("Enter the elements: ");
for (i=0; i<n; ++i)
    scanf("%d", arr[i]);
n = add_beg(arr, n, data);
for (i=0; i<n; i++)
    printf("%d", arr[i]);
}
int add_beg(int arr[], int n, int data){
int i;
for (i=n-1; i>=0; i--) {
    arr[i+1] = arr[i];
}
arr[0] = data;
return n+1;
}
```

When array is full - copying the old array to the new array of size greater than the old array: O(n)

Adding at beginning of array: O(n)
Total time: n+n = 2n = O(n)

or
Copying the old array to the new array of size greater than the old array from index 1: O(n). Add at beg = O(1)
Total time = O(n+1) = O(n)

Scanned with CamScanner

Inserting a node anywhere except start & end

Step 1: Create a new node
" 2: Traversal
" 3: Adding links (Imp first update link part of new node)

Program
```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *link;
};
int main()
{
struct node *head = malloc(sizeof(struct node));
head -> data = 45;
head -> link = NULL;
add_at_end(head, 98);
add_at_end(head, 3);
int data = 67, position = 3;
add_at_pos(head, data, position);
struct node *ptr = head;
while (ptr != NULL) {
printf("%d", ptr -> data);
ptr = ptr -> link;
}
}
```

Scanned with CamScanner

```
void add_at_pos (struct node * head, int data, int pos)
struct node *ptr = head;
struct node *ptr2 = malloc (size of (struct node));
ptr2 -> data = data;
ptr2 -> link = NULL;

pos --;
while (pos != 1) {
ptr2 -> link = ptr -> link;
ptr -> link = ptr2;
}
```

Insertion at certain position (Like V/S array)

LL = O(n) → worst case (n-2) so O(n)

Array - Suppose array is full & insert at pos 5th

arr

2	34	2	6	7	8	90	6	23	39
0	1	2	3	4	5	6	7	8	9

So we create a new array of size (n+1)

arr2

2	34	2	6	78	7	8	90	67	23	39	
0	1	2	3	4	5	6	7	8	9	10	

→ Copy everything upto position 4 & paste it in new array. After that insert new element at position 5 & copy the rest elements from the previous array to the new array.

Scanned with CamScanner

Code

```
#include<stdio.h>
int main(){
  int arr[]={2,34,21,6,7,8,90,23,39};
int pos=5, data=78, i;
int size=sizeof(arr)/sizeof(arr[0]);
int arr2[size+1];

add_at_pos(arr, arr2, size, data, pos);
for(i=0;i<size+1;i++)
{ printf("%d", arr2[i]);
}

Void add_at_pos(int arr[] int arr2[], int n,
    int data, int pos){
  int i;
int index=pos-1;
for(i=0; i<=index-1; i++)
    arr2[i] = arr1[i];
arr2[index]= data;
int j;
for(i=index+1; j=index; i<n+1, j<n;
                          i++ j++)
  arr2[i]=arr[j];
}
```

Time Complexity: O(n)
n → size of original array.

Remember:
1) We can't return an array from a function because an array created in the function is local to that fun

→ This is the reason why arr[2] has been passed as a parameter to a function
→ Passing arr2 (name of the array) is equivalent to passing the address of the first block of arr2. So the changes made in called function will be reflected in the caller function.

Deleting first node of the single linked list.

```
#include<stdio.h>
#include<stdlib.h>
struct node {
    int data;
    struct node *link;
};
int main(){
    head = del_first(head);
    ptr=head;
    while(ptr!=NULL){
    printf("%d" ptr->data);
    ptr=ptr->link;
}}
struct node* del_first(struct node *head){
if(head==NULL)
    printf("list is already empty!");
else{
    struct node *temp=head;
    head=head->link; #node
    free(temp); temp=NULL; //node frees & temp deleted
    return head; }
```

Scanned with CamScanner

Deleting Last Node of Single Linked List

Note that we can access list through head pointer only.

1) Create a pointer temp pointing to head or first node of list
2) The next step is to traverse the list. We will use 2 pointer. One pointer will stop at the last of node of list & other pointer will stop at last node of list.

```
Code #include<stdio.h>
#include<stdlib.h>
struct node{
    int data;
    struct node *link;
};
int main(){
head = del_last(head);
ptr = head;
while(ptr != NULL){
ptr = ptr;
printf("%d", ptr->data);
ptr = ptr->link;
}
}
```

Scanned with CamScanner

```
struct node* del_last(struct node *head)
{
  if(head == NULL)
      printf("List is already empty.");
  else if(head->link == NULL) {
      free(head);
      head = NULL;
  }
  else{ struct node *temp = head;
        struct node *temp2 = head;
        while(temp->link != NULL) {
            temp2 = temp;
            temp = temp->link;
        }
        temp2->link = NULL;
        free(temp);
        temp = NULL; }
}
```

Note - There is no need to return head pointer as we are not updating it in function

Mind Bending Question for you → Can we use a single pointer instead of 2 pointers for deleting the last node of the list?

Deleting the last node by using single pointer
1) Create a pointer temp
2) Traverse it 2 2nd last element
3) free(temp->link).

Scanned with CamScanner

Code

```
#include<stdio.h>
#include<stdlib.h>
struct node{
    int data;
    struct node *link; };
int main(){
  del_list(head);
  ptr=head;
while(ptr!=NULL){
 printf("%d", ptr->data);
}}ptr=ptr->link;
}
void del_list(struct node *head){
 if(head==NULL)
    printf("list is already empty");
 else if(head->link==NULL){
   free(head);
   head=NULL;
}
else{
 struct node *temp=head;
while(temp->link->link!=null)
{temp=temp->link;}
free(temp->link);
temp->link=NULL;
}
```

Scanned with CamScanner

Deletion at end (LL Vs Array)

Linked list: O(N)

Array

arr

23	3	45	12	67	54	6	4
0	1	2	3	4	5	6	7

size = 8

Code

```
#include <stdio.h>
int main() {
    int arr[] = {23, 3, 45, 12, 67, 54, 6, 4};
    int size = sizeof(arr)/sizeof(arr[0]);
    int i;
    size = size - 1;
    for (i=0; i<size; i++)
        printf("%d", arr[i]);
    return 0;
}
```

O(1)

Deleting in Beginning (Array v/s LL)

LL → O(1) already done

Arrays

```
int arr[8];
int n, i;
scanf("%d", &n);
for(i=0; i<n; i++)
    scanf("%d", &arr[i]);
```

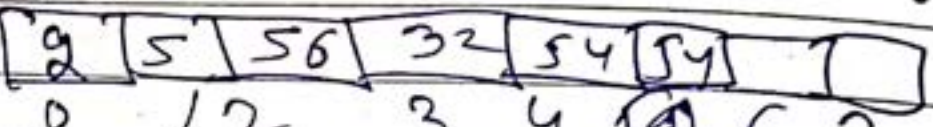
→ Target it to delete this

arr

23	2	5	58	32	54		
0	1	2	3	4	5	6	7

6 6

We shift to left & after shifting



don't traverse this element simply (or) delete at end O(1)

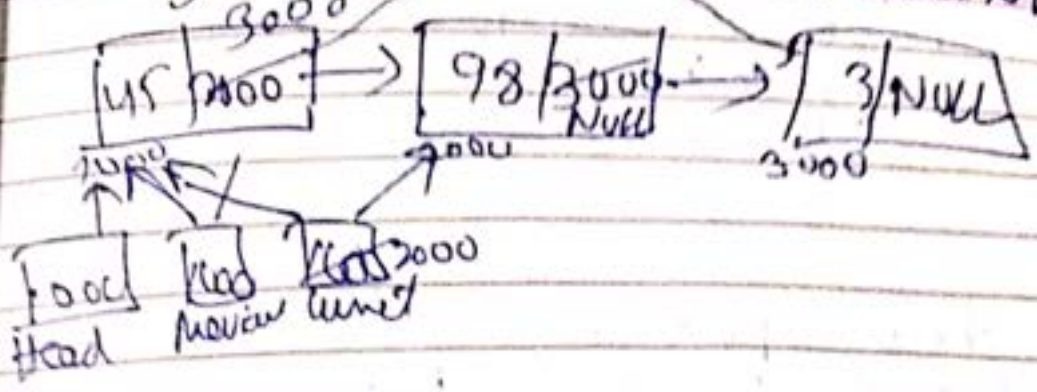
Code

```
int main(){
int arr[8];
int n, i;
printf("Enter the # of elements: ");
scanf("%d", &n);
for(i=0; i<n; i++)
    scanf("%d", &arr[i]);
n=del_first(arr, n);
for(i=0; i<n; i++)
    printf("%d", arr[i]);
return 0;
}
int del_first(int arr[], int n){
int i;
if(n==0){
    printf("Array is empty.");
    return n;
}
for(i=0; i<n-1; i++){
    arr[i]=arr[i+1];
}
return n-1;
}
```

O(n)

Scanned with CamScanner

Deleting node at particular Position



→ Let's say we want to remove node # 2
→ 2 pointers previous & current
→ current → node to be deleted
previous → . . . . // — |
→ then change address.

```
#include<stdio.h> #include<stdlib.h>
struct node { int data; struct node *link;
};
int main(){
int position = 2;
del_pos(&head, position);
ptr = head;
while(ptr != NULL){
printf("%d", ptr->data);
ptr = ptr->link;
}
return 0;
}
void del_pos(struct node **head, int position)
struct node *current = *head;
struct node *previous = *head;
if(*head == NULL)
printf("List is already empty!");
```

Scanned with CamScanner

```
else if(position == 1){
 *head = current->link;
 free(current);
 current = NULL;
}else{

while (position != 1){
previous = current;
current = current->link;
position --;}
previous->link = curr->link;
free(current);
current = NULL;
}}
```

Passing &head in main: if position == 1 then we change head pointer

Deleting the single linked list

Can we delete directly head pointer?
No because 1st node deleted successfully but rest of the nodes are still in memory

So, we need to store the address of the second node somewhere so that we can reach the second node even if the first node is deleted.

Scanned with CamScanner

So we make another pointer pointing at first node & move it to next node before deleting head

i.e temp = temp -> link;
free(head);
head = temp;

```
#include <stdio.h>
#include <stdlib.h>
struct node{
int data;
struct node *link;
};

int main()
{
head = del_list(head);
if(head == NULL)
printf("Linked list deleted successfully");
}
```

Reverse a single linked list -> Our aim is to make the last node of the list the first node - reverse last node of the list) the second node and so on

Create a temp pointer to keep reference of 2nd pointer

```
head -> link = NULL;
temp -> link = head;

temp -> link = head;
```

Scanned with CamScanner

It can be achieved by 2 pointers temp & temp2. So




```
temp2 = head->link;
head->link = temp;
```

The idea is to update the link part of one node at a time pointed by head pointer

But before moving head we store its content in temp.

```
temp2 = head->link;
head->link = temp
temp = head;
head = temp2;
```

At the last node, head becomes NULL. `temp2 = head->link`

So while (head != NULL)

So at end of iteration we write
head = temp;

We can replace temp -> prev
temp2 -> next
to make more sense

```
#include <stdio.h>
#include<stdlib.h>
struct node {int data; struct node* link;};

int main(){
 head = reverse(head);
 ptr = head;
 while (ptr != NULL){
 printf(" %d", ptr->data);
 ptr = ptr->link;
}
struct node* reverse(struct node *head){
struct node *prev = NULL;
  ,  ,  , next = ;
 while (head != NULL){
 next = head->link;
 head->link = prev;
 prev = head;
 head = next; }
head = prev;
 return head;
}
```

Scanned with CamScanner